

Pontificia Universidad Católica del Ecuador

Sede Manabí

Carrera de Ingeniería en Software

Tema:

PortoSinFiltro – Plataforma web de denuncias ciudadanas para Portoviejo

Materia:

Desarrollo de Sistemas de Información

Docente:

Ing. José Luis Naranjo Villota

Autores:

Castro Muñoz Adolfo Agustin

Saltos Macias Elkin Renan

Hernandez Menendez Axel Jhostin

Morales Moreira Frank Marcelo

Medranda Garcia Johan George

Portoviejo, Manabí, Ecuador

2026

DOCUMENTO TÉCNICO DEL PROYECTO

Proyecto: PortoSinFiltro – Plataforma web de denuncias ciudadanas para Portoviejo

Información General

Campo	Detalle
Nombre del Proyecto	PortoSinFiltro
Tipo de Proyecto	Aplicación web (SPA/PWA)
Metodología	Scrum adaptado, con gestión en Jira
Tecnologías	React + Vite, Tailwind CSS, Node.js + Express, Supabase (Auth, PostgreSQL, Storage, Realtime), Vercel
Equipo	5 integrantes
Cliente	Comunidad ciudadana de Portoviejo
Fecha de Inicio	24 de junio de 2026
Enlace de producción	https://porto-sin-filtro.vercel.app

1. Antecedentes

En la actualidad, los habitantes de Portoviejo reportan los problemas de su entorno urbano —baches, alumbrado dañado, acumulación de basura, semáforos apagados o fugas de agua— a través de canales informales como grupos de mensajería, publicaciones en redes sociales o llamadas telefónicas. Esta situación genera dispersión de la información, pérdida de reportes y escasa trazabilidad en su seguimiento.

Frente a esta situación se desarrolló PortoSinFiltro, una plataforma web ciudadana e independiente cuyo propósito es centralizar y, sobre todo, visibilizar los problemas urbanos. A través de un muro público, las denuncias dejan de perderse en canales informales y se exponen ante la comunidad, que puede respaldarlas, complementarlas y darles seguimiento de forma colectiva.

Un factor central es la protección de quien denuncia. Por ello, la plataforma permite publicar de forma anónima: el autor se registra en el sistema y decide si su nombre aparece públicamente o si

su denuncia se muestra como «Ciudadano Anónimo», conservando la identidad real únicamente para fines internos de moderación. De este modo, PortoSinFiltro se articula sobre tres principios: visibilizar los problemas, respaldarlos de forma colectiva y proteger la identidad de quien los reporta.

2. Objetivo General

Se desarrolló una aplicación web denominada PortoSinFiltro, basada en un sistema de denuncias ciudadanas con control de acceso por roles, orientada a visibilizar y dar seguimiento colectivo a los problemas urbanos de la ciudad de Portoviejo mediante un muro público, fortaleciendo la participación ciudadana y la transparencia, y garantizando la seguridad de la información y la protección de la identidad de los denunciantes.

3. Objetivos Específicos

- Se implementó un módulo de seguridad y autenticación basado en roles (ciudadano y administrador) que garantiza la protección de los datos y el control de acceso a las funcionalidades.
- Se desarrolló el registro de denuncias que captura categoría, zona, descripción, evidencia fotográfica y nivel de gravedad, generando automáticamente un titular para el muro público.
- Se incorporó un mecanismo de aportes colaborativos que permite a distintos ciudadanos confirmar, complementar y respaldar una denuncia existente, evitando la duplicación de reportes.
- Se garantizó un modelo de denuncia anónima que protege la identidad del autor frente a terceros y conserva su trazabilidad interna para fines de moderación.
- Se diseñó un muro público que jerarquiza las denuncias según su gravedad y respaldo ciudadano, priorizando los problemas de mayor impacto.
- Se implementó un seguimiento comunitario del estado de cada denuncia (activa, con avance y resuelta) a partir de votos de progreso y confirmaciones de resolución.

- Se generó un panel de estadísticas públicas que muestra la situación de los problemas por categoría, zona y estado, fortaleciendo la transparencia.
- Se aplicaron políticas de control de acceso a nivel de base de datos (RLS) que aseguran la integridad y la confidencialidad de la información.

4. Alcance del Proyecto

El sistema contempla las siguientes acciones según el rol del usuario:

Ciudadano

- Registrarse e iniciar sesión.
- Crear denuncias, públicas o anónimas, con categoría, zona, evidencia fotográfica y nivel de gravedad.
- Apoyar y aportar a las denuncias de otros ciudadanos.
- Marcar el progreso de una denuncia y confirmar su resolución.
- Reportar contenido falso, duplicado u ofensivo.

Administrador

- Moderar el contenido reportado por la comunidad: ocultar y restaurar denuncias.
- Administrar usuarios (activar o inhabilitar cuentas).
- Consultar el panel de administración y las estadísticas.

Visitante

- Consultar el muro público, el listado de denuncias y los rankings sin autenticación.

El sistema es una plataforma ciudadana independiente: no existe integración ni gestión oficial por parte del municipio o cuadrillas. En el código persiste un «Panel municipalidad/cuadrilla» como elemento heredado de una versión anterior, que no forma parte del modelo comunitario actual.

5. Tecnologías Utilizadas

Frontend

Se utilizó React con Vite para la construcción de la interfaz de usuario y Tailwind CSS para el diseño responsivo. La aplicación se implementó como PWA (manifest y service worker) e incorpora mapas mediante Leaflet.

Backend y API

Se desarrolló una API REST con Node.js y Express, con control de acceso opcional (optionalAuth), validación de variables de entorno y limitación de peticiones (rate limiting con express-rate-limit).

Servicios y base de datos (Supabase)

- Auth: autenticación de usuarios y gestión del hash de contraseñas.
- PostgreSQL: base de datos relacional del sistema.
- Storage: almacenamiento de las fotografías de denuncias y aportes.
- Realtime: actualización en vivo del muro público.
- Políticas RLS: control de acceso a nivel de fila.

Despliegue y herramientas

- Vercel para el despliegue en producción (<https://porto-sin-filtro.vercel.app>).
- GitHub para el control de versiones.
- Jira para la gestión del backlog, las historias y las tareas (Scrum).
- GitHub Actions para la integración continua (CI): ejecuta las pruebas de backend y frontend en cada push y pull request a la rama main.
- Figma para el diseño de interfaces y Visual Studio Code como entorno de desarrollo.

6. Metodología de Desarrollo

El proyecto se gestionó con Scrum adaptado, apoyado en el tablero de Jira del proyecto (clave SCRUM). El trabajo se organizó mediante un backlog dividido en ocho épicas, del cual se desprendieron historias, tareas y subtareas asignadas a cada integrante, con seguimiento mediante los estados «Por hacer», «En revisión» y «Finalizado».

7. Roles del Equipo

La siguiente distribución refleja el trabajo real y comprobable de cada integrante en el tablero de Jira y en el repositorio.

Integrante	Área principal	Responsabilidades
Saltos Macias Elkin Renan	Base de datos y autenticación	- Esquema de roles ciudadano/administrador, migraciones y vistas. - Registro e inicio de sesión con Supabase Auth. - Aplicación de migraciones y vistas del esquema comunitario. - PWA (manifest y service worker) y mapa agregado de denuncias.
Hernandez Menendez Axel Jhostin	Frontend de denuncias, comunidad y pruebas	- Formulario, página de detalle y muro público de denuncias. - Acciones comunitarias: apoyo, progreso y resolución. - Realtime del muro (Supabase Realtime). - Pruebas automatizadas (backend y frontend) y de carga (rate limiting). - Paginación y gestión de usuarios en el panel de administración.
Castro Muñoz Adolfo Agustin	Administración, moderación y panel público	- Planificación de la arquitectura. - Panel de administración: ocultar/restaurar denuncias. - Moderación con señales comunitarias (API). - Panel público de solo lectura y su paginación; UI de fotos en aportes.

Integrante	Área principal	Responsabilidades
Morales Moreira Frank Marcelo	Documentación, mapa y demostración	• Documentación del repositorio (README y guías de estados y roles). • Mapa (Leaflet) y subida de fotos a Storage con validación de formatos. • Preparación y ejecución de la demostración en vivo.
Medranda Garcia Johan George	Pruebas, datos y presentación	• Pruebas y QA (dashboard, Plan B standalone, formatos de foto). • Perfiles de usuario y actualización de la base de datos (seed). • Refresco de estado tras acciones comunitarias; guion de la presentación.

8. Arquitectura del Sistema

El sistema se construyó con una arquitectura cliente-servidor por capas. La capa de presentación es una aplicación React + Vite con Tailwind CSS, empaquetada como PWA. La capa de aplicación es una API REST en Node.js + Express que expone los endpoints del sistema, aplica limitación de peticiones (rate limiting) y validación de entorno. Los servicios gestionados de Supabase proporcionan la autenticación, la base de datos PostgreSQL, el almacenamiento de imágenes y la actualización en tiempo real del muro. El despliegue de producción se realizó en Vercel.

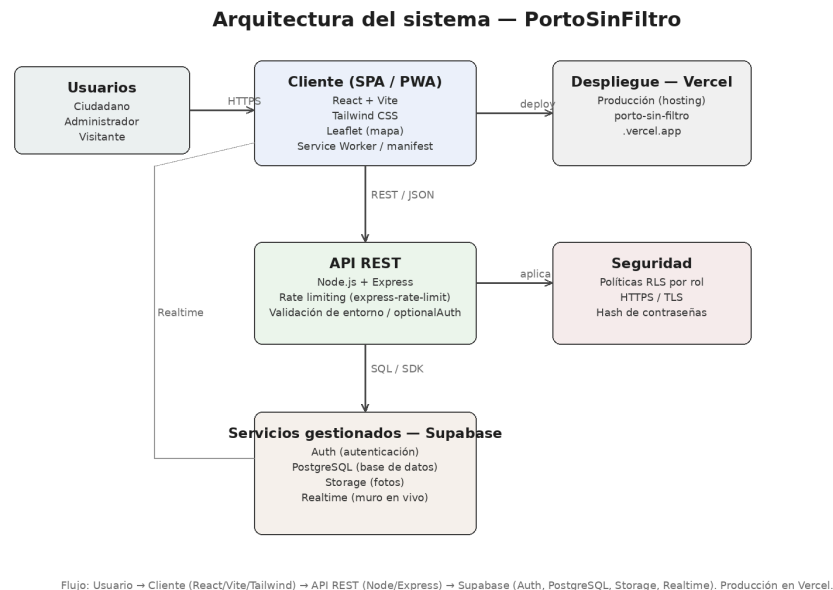


Figura 1. Diagrama de arquitectura del sistema PortoSinFiltro.

9. Módulos del Sistema

El sistema se organizó en los siguientes módulos, correspondientes a las épicas del tablero de Jira:

Módulo 1	Gestión de usuarios y autenticación.
Módulo 2	Registro y gestión de denuncias.
Módulo 3	Muro público.
Módulo 4	Aporte colaborativo.
Módulo 5	Seguimiento ciudadano.
Módulo 6	Anonimato y moderación.
Módulo 7	Estadísticas públicas.
Módulo 8	Infraestructura y configuración técnica.

10. Diagramas de Análisis y Diseño

Esta sección reúne los diagramas de análisis y diseño del sistema exigidos para la documentación: el diagrama de flujo de datos (DFD), los diagramas UML de casos de uso y de clases, y el modelo entidad-relación (DER).

10.1 Diagrama de Flujo de Datos (DFD)

El DFD de contexto presenta al sistema como un único proceso que interactúa con los actores externos (ciudadano, visitante y administrador). El DFD de nivel 1 descompone ese proceso en los subprocesos del sistema y sus almacenes de datos.

DFD — Nivel de contexto (Diagrama 0)



Figura 2. DFD de nivel de contexto.

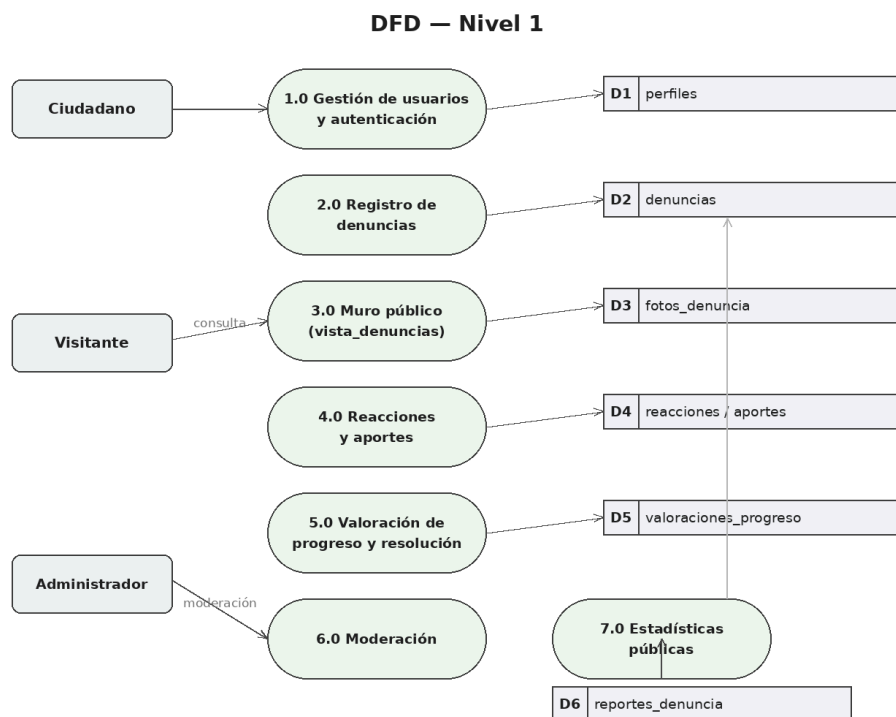


Figura 3. DFD de nivel 1.

10.2 Diagrama de Casos de Uso (UML)

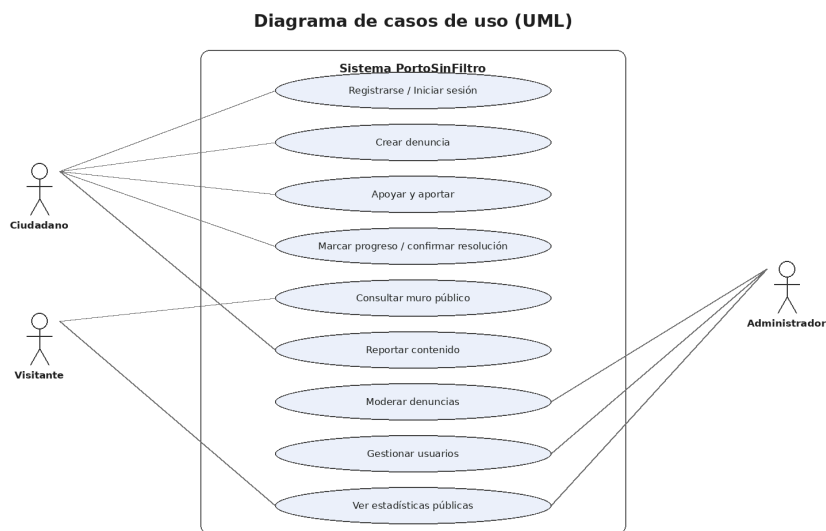
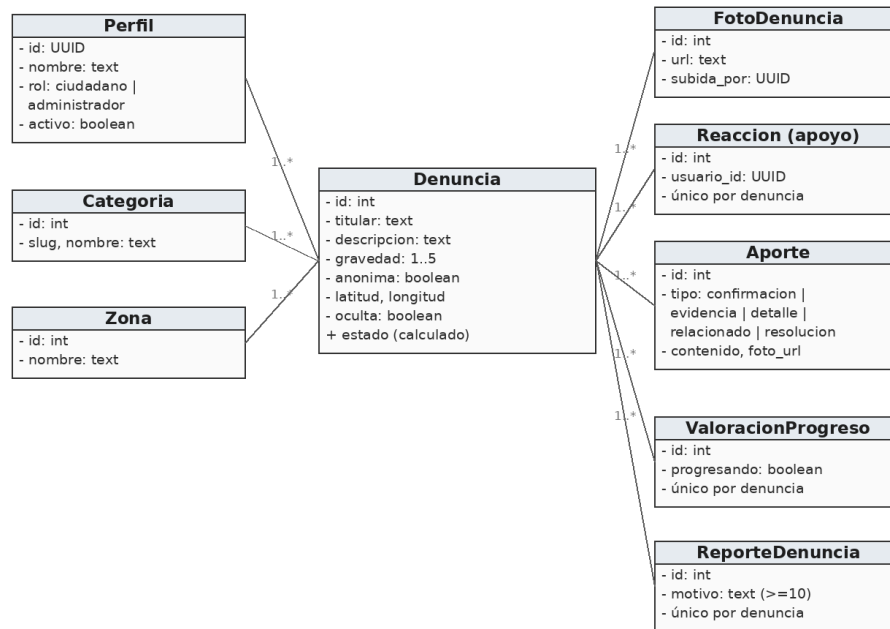


Figura 4. Diagrama de casos de uso.

10.3 Diagrama de Clases (UML)

Diagrama de clases (UML)

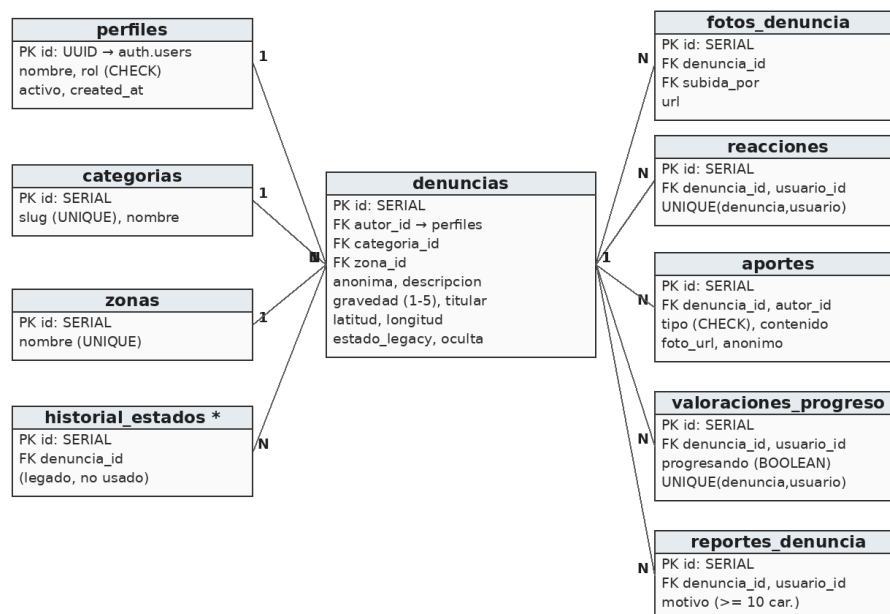


El estado (activa / con_avance / resuelta) se deriva en vista_denuncias a partir de los aportes de tipo resolución y de las valoraciones de progreso.

Figura 5. Diagrama de clases.

10.4 Modelo Entidad-Relación (DER)

Modelo Entidad-Relación (DER)



* Tabla heredada: se conserva en el esquema, sin uso en el modelo comunitario actual.

Figura 6. Modelo entidad-relación.

11. Casos de Uso

Se describen los principales casos de uso del sistema con su actor, precondition, flujo principal y postcondición.

CU-01: Registrarse / Iniciar sesión

Actor: Ciudadano

Precondición: El usuario no está autenticado.

Flujo principal: Ingresa su correo y contraseña; el sistema valida las credenciales con Supabase Auth y carga la interfaz de su rol.

Postcondición: El usuario queda autenticado como ciudadano.

CU-02: Crear denuncia

Actor: Ciudadano

Precondición: El ciudadano tiene la sesión iniciada.

Flujo principal: Completa categoría, zona, descripción, gravedad y fotografías, y elige si es anónima; el sistema genera el titular y guarda la denuncia en estado «activa».

Postcondición: La denuncia queda publicada en el muro público.

CU-03: Apoyar y aportar

Actor: Ciudadano

Precondición: Existe al menos una denuncia publicada.

Flujo principal: Apoya la denuncia (una sola vez) o agrega un aporte con comentario o evidencia.

Postcondición: Se actualiza el respaldo y la información de la denuncia.

CU-04: Marcar progreso / confirmar resolución

Actor: Ciudadano

Precondición: La denuncia está activa.

Flujo principal: Marca que la denuncia presenta avance o confirma su resolución; el sistema recalcula el estado según la participación de la comunidad.

Postcondición: El sistema recalcula el estado y lo actualiza a «con_avance» o «resuelta» únicamente cuando se cumplen los umbrales comunitarios.

CU-05: Consultar muro público

Actor: Visitante

Precondición: Ninguna.

Flujo principal: Accede al muro, aplica filtros y ordenamientos y revisa las estadísticas públicas.

Postcondición: La información se visualiza sin necesidad de autenticación.

CU-06: Moderar y gestionar usuarios

Actor: Administrador

Precondición: El administrador tiene la sesión iniciada.

Flujo principal: Revisa los reportes de la comunidad, oculta o restaura denuncias e inhabilita o activa cuentas de usuario.

Postcondición: El contenido queda moderado y los usuarios gestionados.

12. Historias de Usuario

ID	Historia de usuario	Criterio de aceptación
HU-01	Como ciudadano, quiero registrarme e iniciar sesión, para crear y seguir mis denuncias.	Puedo autenticarme y el sistema carga mi interfaz de ciudadano.
HU-02	Como ciudadano, quiero crear una denuncia con foto, zona y gravedad, para reportar un problema.	La denuncia aparece en el muro con su titular en estado activa.
HU-03	Como ciudadano, quiero publicar de forma anónima, para reportar sin exponer mi identidad.	Mi nombre no se muestra al público; solo el administrador puede verlo.
HU-04	Como ciudadano, quiero apoyar y aportar a una denuncia, para respaldarla y evitar duplicados.	Mi apoyo cuenta una sola vez y suma al ranking.

ID	Historia de usuario	Criterio de aceptación
HU-05	Como ciudadano, quiero marcar progreso y confirmar la resolución, para reflejar el avance real.	El estado pasa a con_avance cuando existen al menos dos valoraciones con progresando = true y estas superan a las negativas; pasa a resuelta cuando existen al menos tres aportes de tipo resolución de usuarios distintos.
HU-06	Como visitante, quiero consultar el muro y las estadísticas, para informarme sin registrarme.	Veo el muro, el listado y los rankings sin iniciar sesión.
HU-07	Como ciudadano, quiero reportar contenido falso u ofensivo, para mantener la calidad del muro.	El reporte queda registrado para su moderación.
HU-08	Como administrador, quiero ocultar o restaurar denuncias e inhabilitar usuarios, para moderar la plataforma.	El contenido reportado puede ocultarse y restaurarse.

13. Requisitos Funcionales (RF)

Los requisitos funcionales describen las acciones que ejecuta el sistema para satisfacer las necesidades de los usuarios.

Módulo 1: Gestión de Usuarios y Autenticación

- **RF-01: Registro de Usuarios:** el sistema permite el registro de ciudadanos mediante correo y contraseña; el usuario administrador se crea directamente en la base de datos.
- **RF-02: Autenticación:** el sistema permite iniciar y cerrar sesión con credenciales gestionadas por Supabase Auth, cargando la interfaz correspondiente a cada rol.
- **RF-03: Control de Acceso por Rol (RBAC):** el sistema restringe las interfaces y operaciones según el rol (visitante, ciudadano y administrador), reforzado con políticas RLS.

Módulo 2: Registro y Gestión de Denuncias

- **RF-04: Creación de Denuncias:** el ciudadano registra una denuncia con categoría, descripción, fotografías, zona, gravedad y opción de anonimato.
- **RF-05: Generación Automática de Titular:** al guardarse, el sistema genera un titular a partir de la categoría y la zona, e inicia la denuncia en estado «activa».

Módulo 3: Muro Público

- **RF-06: Muro Público de Denuncias:** el sistema muestra el muro con las denuncias, un problema destacado, el contador de días sin resolver y un ranking por respaldo.
- **RF-07: Actualización en Tiempo Real:** el muro se actualiza mediante Supabase Realtime, escuchando los cambios de las tablas denuncias, reacciones, aportes, fotos_denuncia y valoraciones_progreso.

El funcionamiento de esta característica requiere ejecutar previamente la migración `database/migracion_realtime.sql` en Supabase, que habilita las políticas de lectura y agrega las tablas a la publicación `supabase_realtime`. Debe verificarse su aplicación en el entorno de producción antes de la demostración.

Módulo 4: Aporte Colaborativo

- **RF-08: Apoyo a Denuncias:** el ciudadano apoya una denuncia (una vez por denuncia), elevando su posición en el ranking.
- **RF-09: Aportes Colaborativos:** el ciudadano confirma, aporta evidencia o agrega detalle a una denuncia existente, evitando duplicados.

Módulo 5: Seguimiento Ciudadano

- **RF-10: Valoración de Progreso:** cualquier ciudadano emite una valoración de progreso (una por denuncia); el estado «con_avance» se calcula a partir de las valoraciones de la comunidad.
- **RF-11: Confirmación de Resolución:** la resolución se registra mediante un aporte de tipo `resolucion`, con un máximo de uno por ciudadano y denuncia; la denuncia pasa a «resuelta» al reunir tres aportes de este tipo de usuarios distintos.

Los estados de una denuncia son activa, con avance (con_avance) y resuelta. No se almacenan como columna: se calculan en la vista vista_denuncias a partir de las valoraciones de progreso y de los aportes de tipo resolucion, según los umbrales indicados. El sistema no incluye un módulo de notificaciones.

Módulo 6: Anonimato y Moderación

- **RF-12: Denuncias Anónimas:** el autor publica como «Ciudadano Anónimo»; su identidad solo es visible para el administrador con fines de moderación.
- **RF-13: Reporte de Contenido:** el ciudadano reporta denuncias falsas, duplicadas u ofensivas.
- **RF-14: Moderación y Gestión de Usuarios:** el administrador oculta o restaura denuncias y activa o inhabilita cuentas de usuario.

Módulo 7: Estadísticas Públicas

- **RF-15: Estadísticas Públicas:** el sistema presenta estadísticas por categoría, zona y estado, accesibles públicamente sin exponer datos sensibles.

Módulo 8: Búsqueda y Visualización

- **RF-16: Visualización Pública:** cualquier visitante consulta el muro y el listado de denuncias sin autenticación.
- **RF-17: Búsqueda y Filtros:** el sistema permite filtrar las denuncias por categoría, zona, gravedad y estado, con ordenamiento por respaldo o antigüedad.

14. Requisitos No Funcionales (RNF)

Seguridad

- **RNF-01: Cifrado en Tránsito:** la comunicación entre el cliente y los servicios se realiza mediante HTTPS con cifrado TLS.
- **RNF-02: Protección de Persistencia:** el acceso a las tablas de PostgreSQL se restringe mediante políticas RLS según el rol del usuario.

- **RNF-03: Resguardo de Contraseñas:** las contraseñas se gestionan mediante Supabase Auth y nunca se almacenan en texto plano.
- **RNF-04: Protección de la Identidad Anónima:** la identidad real de un autor anónimo solo es accesible para el administrador, con fines de moderación.
- **RNF-05: Limitación de Peticiones:** la API aplica rate limiting (express-rate-limit) para mitigar el abuso y la sobrecarga.

Rendimiento y Usabilidad

- **RNF-06: Optimización de Carga:** el frontend se empaqueta con Vite y se sirve como PWA para mejorar el tiempo de carga.
- **RNF-07: Diseño Responsivo:** la interfaz, construida con Tailwind CSS, se adapta a dispositivos móviles, tabletas y computadoras de escritorio.

Confiabilidad y Mantenibilidad

- **RNF-08: Integridad de los Datos:** PostgreSQL garantiza las propiedades ACID en las operaciones sobre las denuncias y sus estados.
- **RNF-09: Compatibilidad de Navegadores:** la aplicación funciona en las versiones recientes de Chrome, Firefox, Edge y Safari.
- **RNF-10: Modularidad del Código:** el código se organiza en componentes reutilizables y módulos, con control de versiones en GitHub.
- **RNF-11: Integración Continua:** cada push y pull request a la rama main ejecuta automáticamente las pruebas de backend y frontend mediante GitHub Actions.

Uso Responsable

- **RNF-12: Responsabilidad del Contenido:** la plataforma trata problemas de infraestructura y servicios, no señala a personas, y permite la moderación del contenido.

15. Modelo de Datos

Modelo relacional implementado sobre PostgreSQL (Supabase). Las definiciones corresponden al archivo database/BDD.sql del repositorio. La tabla perfiles extiende auth.users de Supabase; el resto de las tablas emplea identificadores SERIAL. El estado comunitario de una denuncia no se almacena como columna: se calcula en la vista vista_denuncias.

15.1 perfiles

Campo	Tipo	Descripción
id	UUID (PK)	Referencia a auth.users(id); ON DELETE CASCADE.
nombre	TEXT	NOT NULL. Nombre del usuario.
rol	TEXT (CHECK)	NOT NULL. Valores: ciudadano, administrador.
activo	BOOLEAN	NOT NULL, DEFAULT true.
created_at	TIMESTAMPTZ	NOT NULL, DEFAULT NOW().

15.2 categorias

Campo	Tipo	Descripción
id	SERIAL (PK)	Identificador de la categoría.
slug	TEXT (UNIQUE)	NOT NULL. Ej.: baches_vias, alumbrado, basura.
nombre	TEXT	NOT NULL. Nombre visible de la categoría.

15.3 zonas

Campo	Tipo	Descripción
id	SERIAL (PK)	Identificador de la zona.
nombre	TEXT (UNIQUE)	NOT NULL. Sector o parroquia de Portoviejo.

15.4 denuncias

Campo	Tipo	Descripción
id	SERIAL (PK)	Identificador de la denuncia.
autor_id	UUID (FK perfiles)	NOT NULL. Autor real; se conserva aunque sea anónima.
anonima	BOOLEAN	NOT NULL, DEFAULT false. Oculta el autor al público.
categoria_id	INTEGER (FK categorias)	NOT NULL.
zona_id	INTEGER (FK zonas)	NOT NULL.
descripcion	TEXT	NOT NULL. Descripción del problema.
gravedad	SMALLINT (CHECK)	NOT NULL. Valor entre 1 y 5.
latitud / longitud	DOUBLE PRECISION	Ubicación opcional para el mapa.
titular	TEXT	NOT NULL. Generado automáticamente en el backend.
estado_legacy	TEXT	DEFAULT 'activa'. Columna histórica, no usada por la aplicación.
oculta	BOOLEAN	NOT NULL, DEFAULT false. Moderación del administrador.
created_at / updated_at	TIMESTAMPTZ	NOT NULL. updated_at se actualiza por trigger.

15.5 fotos_denuncia

Campo	Tipo	Descripción
id	SERIAL (PK)	Identificador de la foto.
denuncia_id	INTEGER (FK denuncias)	NOT NULL. ON DELETE CASCADE.
url	TEXT	NOT NULL. Ruta pública en Supabase Storage.

Campo	Tipo	Descripción
subida_por	UUID (FK perfiles)	NOT NULL. Usuario que subió la imagen.
created_at	TIMESTAMPTZ	NOT NULL, DEFAULT NOW().

15.6 reacciones

Campo	Tipo	Descripción
id	SERIAL (PK)	Identificador de la reacción de apoyo.
denuncia_id	INTEGER (FK denuncias)	NOT NULL. ON DELETE CASCADE.
usuario_id	UUID (FK perfiles)	NOT NULL.
UNIQUE	(denuncia_id, usuario_id)	Un apoyo por usuario y denuncia.
created_at	TIMESTAMPTZ	NOT NULL, DEFAULT NOW().

15.7 aportes

Campo	Tipo	Descripción
id	SERIAL (PK)	Identificador del aporte.
denuncia_id	INTEGER (FK denuncias)	NOT NULL. ON DELETE CASCADE.
autor_id	UUID (FK perfiles)	NOT NULL.
anonimo	BOOLEAN	NOT NULL, DEFAULT false.
tipo	TEXT (CHECK)	confirmacion, evidencia, detalle, relacionado, resolucion.
contenido	TEXT	Texto del aporte.
foto_url	TEXT	Evidencia fotográfica opcional.
índice único	(denuncia_id, autor_id)	Solo para tipo = resolucion: una por usuario.

15.8 valoraciones_progreso

Campo	Tipo	Descripción
id	SERIAL (PK)	Identificador de la valoración.
denuncia_id	INTEGER (FK denuncias)	NOT NULL. ON DELETE CASCADE.
usuario_id	UUID (FK perfiles)	NOT NULL.
progresando	BOOLEAN	NOT NULL. true indica avance; false, que sigue igual.
UNIQUE	(denuncia_id, usuario_id)	Una valoración por usuario y denuncia.
created_at / updated_at	TIMESTAMPTZ	NOT NULL, DEFAULT NOW().

15.9 reportes_denuncia

Campo	Tipo	Descripción
id	SERIAL (PK)	Identificador del reporte.
denuncia_id	INTEGER (FK denuncias)	NOT NULL. ON DELETE CASCADE.
usuario_id	UUID (FK perfiles)	NOT NULL.
motivo	TEXT (CHECK)	NOT NULL. Mínimo 10 caracteres.
UNIQUE	(denuncia_id, usuario_id)	Un reporte por usuario y denuncia.

15.10 historial_estados (tabla heredada)

Se conserva en el esquema por compatibilidad, pero no se utiliza en el modelo comunitario actual: registraba los cambios de estado y la respuesta oficial cuando el proyecto contemplaba la gestión por cuadrillas.

15.11 Vistas y cálculo del estado

El esquema define las vistas vista_denuncias (excluye las denuncias ocultas) y vista_denuncias_admin (las incluye, para moderación). Ambas enriquecen la denuncia con su

categoría, zona, autor —o «Ciudadano Anónimo» cuando corresponde—, los días sin resolver, la foto de portada y los totales de apoyos, aportes, fotos, valoraciones y reportes.

El estado se calcula así: la denuncia es «resuelta» cuando acumula tres o más aportes de tipo resolución; es «con_avance» cuando registra al menos dos valoraciones con progresando = true y estas superan a las negativas; en cualquier otro caso permanece «activa».

16. Diccionario de Datos

Resumen de los campos del esquema con su tipo y restricción, según database/BDD.sql. PK: clave primaria; FK: clave foránea; NN: no nulo; U: único.

Tabla	Campo	Tipo	Restricción / Descripción
perfiles	id	UUID	PK, FK a auth.users(id).
perfiles	nombre	TEXT	NN.
perfiles	rol	TEXT	NN, CHECK: ciudadano administrador.
perfiles	activo	BOOLEAN	NN, DEFAULT true.
categorias	id	SERIAL	PK.
categorias	slug	TEXT	NN, U. Identificador técnico.
categorias	nombre	TEXT	NN.
zonas	id	SERIAL	PK.
zonas	nombre	TEXT	NN, U.
denuncias	id	SERIAL	PK.
denuncias	autor_id	UUID	NN, FK a perfiles.
denuncias	anonima	BOOLEAN	NN, DEFAULT false.
denuncias	categoria_id	INTEGER	NN, FK a categorias.
denuncias	zona_id	INTEGER	NN, FK a zonas.
denuncias	descripcion	TEXT	NN.
denuncias	gravedad	SMALLINT	NN, CHECK entre 1 y 5.

Tabla	Campo	Tipo	Restricción / Descripción
denuncias	latitud / longitud	DOUBLE PRECISION	Opcional (mapa).
denuncias	titular	TEXT	NN. Autogenerado.
denuncias	estado_legal y	TEXT	DEFAULT 'activa'. No usado por la aplicación.
denuncias	oculta	BOOLEAN	NN, DEFAULT false.
denuncias	updated_at	TIMESTAMPZ	NN. Trigger actualizar_updated_at.
fotos_denuncia	denuncia_id	INTEGER	NN, FK a denuncias (CASCADE).
fotos_denuncia	url	TEXT	NN.
fotos_denuncia	subida_por	UUID	NN, FK a perfiles.
reacciones	denuncia_id	INTEGER	NN, FK a denuncias (CASCADE).
reacciones	usuario_id	UUID	NN, FK a perfiles.
reacciones	(denuncia_id , usuario_id)	UNIQUE	Un apoyo por usuario y denuncia.
aportes	denuncia_id	INTEGER	NN, FK a denuncias (CASCADE).
aportes	autor_id	UUID	NN, FK a perfiles.
aportes	anonimo	BOOLEAN	NN, DEFAULT false.
aportes	tipo	TEXT	NN, CHECK: confirmacion evidencia detalle relacionado resolucion.
aportes	contenido / foto_url	TEXT	Opcionales.
valoraciones_progreso	denuncia_id	INTEGER	NN, FK a denuncias (CASCADE).
valoraciones_progreso	usuario_id	UUID	NN, FK a perfiles.

Tabla	Campo	Tipo	Restricción / Descripción
valoraciones_progreso	progresando	BOOLEAN	NN. true = hay avance.
valoraciones_progreso	(denuncia_id, usuario_id)	UNIQUE	Una valoración por usuario.
reportes_denuncia	denuncia_id	INTEGER	NN, FK a denuncias (CASCADE).
reportes_denuncia	usuario_id	UUID	NN, FK a perfiles.
reportes_denuncia	motivo	TEXT	NN, CHECK: mínimo 10 caracteres.
reportes_denuncia	(denuncia_id, usuario_id)	UNIQUE	Un reporte por usuario.

Todas las tablas incluyen `created_at (TIMESTAMP TZ NOT NULL DEFAULT NOW())`. El estado de la denuncia no es una columna: se deriva en las vistas `vista_denuncias` y `vista_denuncias_admin`.

17. Entregables

Categoría	Elementos entregables
Documentación	• Documento técnico del proyecto. • Historias de usuario y casos de uso. • Diagrama de arquitectura. • Modelo de datos (DER/MR) y archivo BDD.sql. • Manual técnico y manual de usuario.
Software	• Código fuente del frontend (React + Vite) y del backend (Node + Express). • Base de datos en Supabase (PostgreSQL) y script BDD.sql. • Aplicación funcional desplegada en Vercel. • Datos de prueba.
Pruebas	• Pruebas automatizadas de backend y frontend. • Script de prueba de rate limiting bajo carga. • Flujo de integración continua (.github/workflows/ci.yml).

Presentación	Exposición y demostración en vivo del sistema.
--------------	--

18. Riesgos Identificados

Riesgo	Impacto	Mitigación
Retraso en tareas	Alto	Seguimiento semanal en el tablero de Jira.
Problemas de integración entre módulos	Alto	Integración frecuente del código y pruebas sobre una base de datos compartida.
Pérdida de código	Alto	Uso obligatorio de GitHub.
Configuración incorrecta de políticas RLS	Alto	Pruebas de seguridad por cada rol.
Contenido falso, duplicado u ofensivo	Medio	Reportes de la comunidad y moderación del administrador.
Errores en la base de datos	Medio	Pruebas constantes con datos de prueba.

19. Criterios de Éxito

El proyecto se considera exitoso al cumplir los siguientes criterios:

1. Un ciudadano puede registrarse y crear una denuncia con categoría, foto, zona y gravedad.
2. Un ciudadano puede publicar una denuncia anónima y su nombre no se muestra a los demás.
3. Otro ciudadano puede apoyar y aportar a una denuncia existente.
4. La denuncia aparece en el muro público con su titular y su contador de días.
5. El estado de la denuncia (activa, con avance, resuelta) se determina con la participación de la comunidad.
6. Los ciudadanos pueden reportar contenido y el administrador puede ocultarlo o restaurarlo.
7. El sistema muestra estadísticas públicas por categoría, zona y estado.

8. Un visitante sin sesión visualiza el muro público, pero no puede interactuar.
9. Las políticas RLS impiden accesos indebidos entre roles, incluyendo la protección de la identidad anónima.
10. La aplicación funciona de forma integrada y está desplegada en producción.

20. Exclusiones del Proyecto

Con el fin de delimitar el alcance, no forman parte del sistema las siguientes funcionalidades:

- Integración o entrega del sistema al GAD Municipal de Portoviejo o a cualquier entidad pública; PortoSinFiltro es una plataforma ciudadana independiente.
- Gestión, asignación o resolución oficial de las denuncias por parte de autoridades o cuadrillas (el «Panel municipalidad/cuadrilla» del código es un elemento heredado y no usado).
- Módulo de notificaciones; el envío de correos permanece desactivado (`NOTIFICATIONS_ENABLED = false`) y no forma parte de la entrega.
- Aplicación móvil nativa para Android o iOS.
- Verificación de identidad oficial de los ciudadanos (cédula o datos biométricos).
- Chat en tiempo real entre usuarios y análisis predictivo mediante inteligencia artificial.

21. Manual Técnico

21.1 Requisitos

- Node.js y npm instalados.
- Cuenta y proyecto en Supabase (Auth, PostgreSQL, Storage).
- Cuenta en Vercel para el despliegue.
- Navegador web moderno.

21.2 Estructura del proyecto

El repositorio se organiza en dos partes: el frontend (React + Vite + Tailwind CSS) y el backend (API REST en Node.js + Express). La base de datos y los servicios se administran desde Supabase.

21.3 Variables de entorno

- Frontend: VITE_SUPABASE_URL y VITE_SUPABASE_ANON_KEY.
- Backend: SUPABASE_URL y SUPABASE_SERVICE_KEY (clave secreta; nunca se expone al frontend).
- Backend: FRONTEND_URL con los orígenes permitidos por CORS, separados por coma, y PORT.
- Backend: NOTIFICATIONS_ENABLED se mantiene en false (el envío de correos no forma parte de la entrega).

21.4 Instalación y ejecución

- Clonar el repositorio desde GitHub.
- Instalar dependencias con «npm install» en el frontend y en el backend.
- Configurar las variables de entorno en cada parte.
- Ejecutar database/BDD.sql en el editor SQL de Supabase: crea el esquema, los catálogos de categorías y zonas, las vistas, las políticas RLS y el bucket de Storage.
- Aplicar las migraciones de database/ que correspondan (por ejemplo, migracion_realtime.sql para el muro en tiempo real).
- Levantar el backend y el frontend con «npm run dev» (en terminales separadas).

21.5 Despliegue y pruebas

El script BDD.sql crea el esquema y los catálogos, pero no los usuarios ni los datos de prueba: las cuentas se registran desde la aplicación (el trigger handle_new_user crea el perfil con rol ciudadano) y el rol administrador se asigna manualmente en Supabase.

El despliegue de producción se realiza en Vercel, con la configuración de CORS y las variables de entorno correspondientes; la aplicación está disponible en <https://porto-sin-filtro.vercel.app>.

Las pruebas automatizadas de backend y frontend se ejecutan con el comando de pruebas del proyecto, e incluyen un script de prueba de rate limiting bajo carga.

22. Manual de Usuario

22.1 Ciudadano

1. Registrarse e iniciar sesión con correo y contraseña.
2. Crear una denuncia: elegir categoría y zona, describir el problema, subir fotografías, indicar la gravedad y decidir si es anónima.
3. Consultar el muro público y apoyar las denuncias con las que se está de acuerdo.
4. Aportar a una denuncia existente con un comentario o evidencia adicional.
5. Marcar el progreso de una denuncia o confirmar su resolución.
6. Reportar contenido falso, duplicado u ofensivo.

22.2 Visitante

1. Consultar el muro público, el listado de denuncias y los rankings sin necesidad de registrarse.
2. Revisar las estadísticas públicas.

22.3 Administrador

1. Revisar los reportes de la comunidad.
2. Ocultar o restaurar denuncias.
3. Activar o inhabilitar cuentas de usuario.
4. Consultar el panel de administración y las estadísticas.

23. Evidencias y Enlaces

Aplicación en producción	https://porto-sin-filtro.vercel.app (captura en el Anexo A).
Tablero Jira (Scrum)	https://codificandote.atlassian.net — proyecto SCRUM (portosinfiltro). Capturas en el Anexo A.

Repositorio de código	https://github.com/castro-bot/portoSinFiltro.git
Diagrama de arquitectura	Incluido en la sección 8 (Figura 1).
Modelo de datos / BDD.sql	Sección 15; el esquema definitivo se encuentra en database/BDD.sql.
Pruebas automatizadas	Backend (28 pruebas) y frontend (20 pruebas) con Vitest; incluye la prueba de rate limiting. Capturas en el Anexo A.
Integración continua (CI)	GitHub Actions: .github/workflows/ci.yml ejecuta las pruebas de backend y frontend en cada push y pull request a main. Captura en el Anexo A.
Despliegue	Frontend y backend en Vercel, con configuración de producción (CORS y variables de entorno).

Nota: las notificaciones por correo se mantienen desactivadas mediante la variable NOTIFICATIONS_ENABLED y no se presentan como funcionalidad entregada.

Anexo A. Evidencias del Sistema

Este anexo reúne las capturas que respaldan el funcionamiento del sistema, la ejecución de las pruebas automatizadas, la integración continua y la gestión del proyecto en Jira.

A.1 Aplicación en producción

El sistema se encuentra desplegado y accesible públicamente en <https://porto-sin-filtro.vercel.app>. El muro público muestra las denuncias registradas, los filtros por estado (ACTIVA, CON AVANCE, RESUELTA), el ordenamiento por recientes, apoyos y gravedad, y los selectores de zona y categoría.

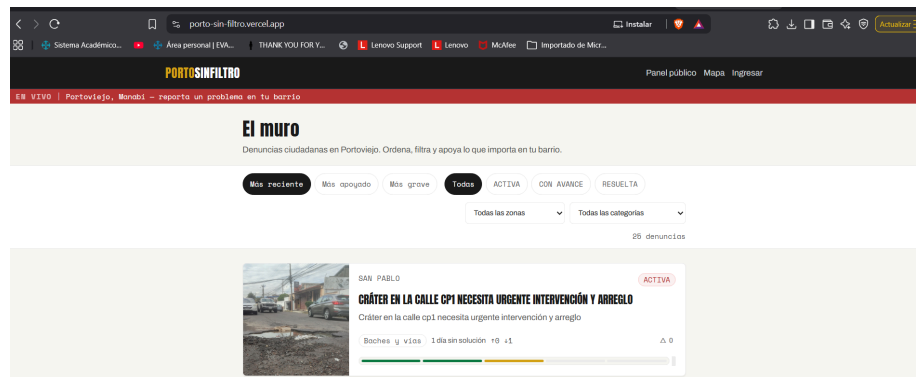


Figura 7. Muro público de la aplicación en producción.

A.2 Integración continua (GitHub Actions)

El repositorio ejecuta automáticamente el flujo de integración continua definido en `.github/workflows/ci.yml` con cada push y pull request a la rama main. La totalidad de las ejecuciones registradas finalizó de forma satisfactoria.

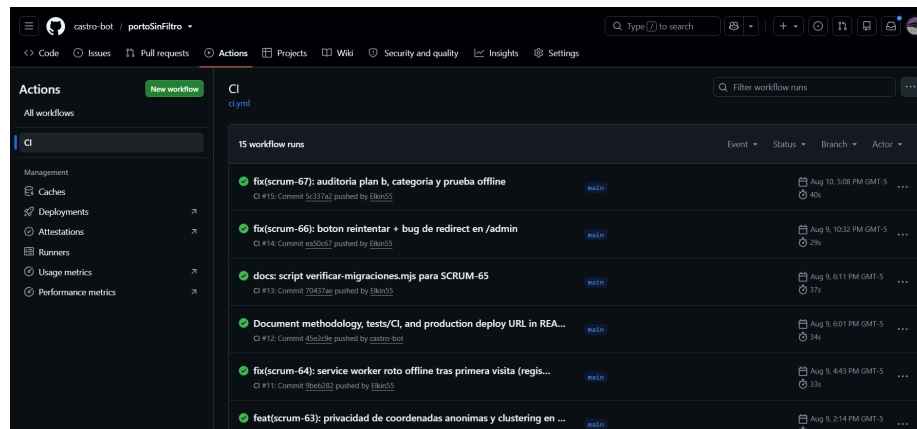


Figura 8. Ejecuciones del flujo de integración continua en GitHub Actions.

A.3 Pruebas automatizadas del backend

La suite del backend se ejecuta con Vitest mediante el comando `npm test`. Comprende siete archivos de prueba y veintiocho casos, que cubren la autenticación, el panel de administración, la gestión de usuarios, el dashboard, las denuncias, el estado de salud del servicio y la limitación de peticiones. La prueba de rate limiting verifica que la API responde con el código 429 al superar las cien peticiones globales en quince minutos.

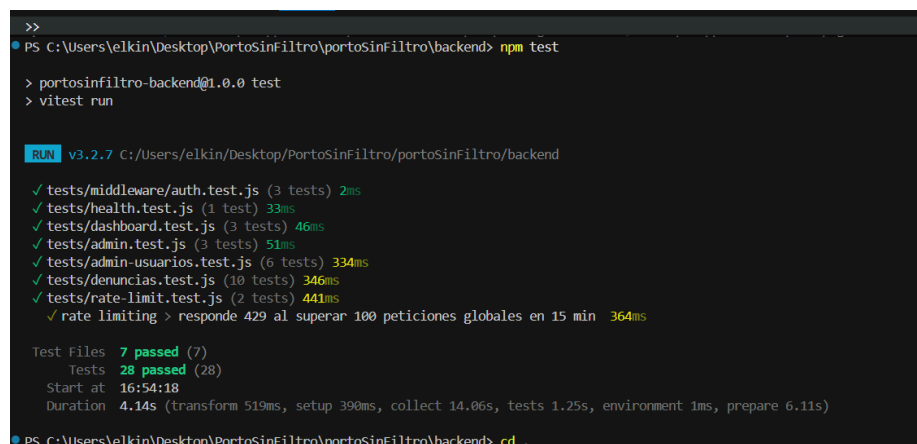


Figura 9. Ejecución de las pruebas automatizadas del backend (28 pruebas superadas).

A.4 Pruebas automatizadas del frontend

La suite del frontend, también con Vitest, comprende seis archivos y veinte casos de prueba, entre ellos los de la lógica de estado comunitario (estadoComunitario), la suscripción al muro en tiempo real (useMuroRealtime) y los componentes de interfaz de barra de gravedad, paginación y tarjeta de denuncia.

```

RUN v3.2.7 C:/Users/elkin/Desktop/PortoSinfiltro/portoSinfiltro/frontend

✓ src/lib/constants.test.js (4 tests) 5ms
✓ src/lib/estadoComunitario.test.js (4 tests) 4ms
✓ src/components/ui/BarraGravedad.test.jsx (3 tests) 126ms
✓ src/components/ui/Paginacion.test.jsx (3 tests) 155ms
✓ src/components/ui/DenunciaCard.test.jsx (4 tests) 155ms
(node:35452) ExperimentalWarning: localStorage is not available because --localstorage-file was not provided.
(Use `node --trace-warnings ...` to show where the warning was created)
✓ src/lib/useNuroRealtime.test.js (2 tests) 3ms

Test Files 6 passed (6)
Tests 20 passed (20)
Start at 16:55:49
Duration 20.82s (transform 538ms, setup 25.19s, collect 2.17s, tests 448ms, environment 86.15s, prepare 3.22s)

```

Figura 10. Ejecución de las pruebas automatizadas del frontend (20 pruebas superadas).

A.5 Gestión del proyecto en Jira

El proyecto se gestionó en el tablero Jira portosinfiltro (clave SCRUM), donde se registran las actividades con su responsable, informador y estado. La vista de lista permite verificar la distribución del trabajo entre los cinco integrantes descrita en la sección 7.

Actividad	Persona asignada	Informador	Prioridad	Estado
SCRUM-3 Prueba testin dashboard	JOHAN GEORGE ME...	Jose Luis Naran...	Ninguno	Finalizado
SCRUM-5 Creacion de perfiles de usuario	JOHAN GEORGE ME...	Jose Luis Naran...	Medium	Finalizado
SCRUM-8 Inicializacion del codigo y repositorio en Git...	AXEL JHOSTIN HER...	AXEL JHOSTIN ...	Medium	Finalizado
SCRUM-9 Login y registro funcional	Sin asignar	AXEL JHOSTIN ...	Medium	Finalizado
SCRUM-10 Arreglar diseño de componentes	Sin asignar	AXEL JHOSTIN ...	Medium	Por hacer
SCRUM-11 Base de datos actualizada	JOHAN GEORGE ME...	AXEL JHOSTIN ...	Medium	En revisión
SCRUM-15 Refrescar estado tras acciones comunitari...	JOHAN GEORGE ME...	ELKIN RENAN S...	Medium	Finalizado
SCRUM-16 Desarrollo del formulario para crear denuncias	AXEL JHOSTIN HFR	Ardolfo Castro	Medium	Finalizado

Figura 11. Vista de lista de actividades en Jira con responsables y estados.

El backlog concentra las actividades pendientes y en revisión, y permite dar seguimiento al avance del proyecto.

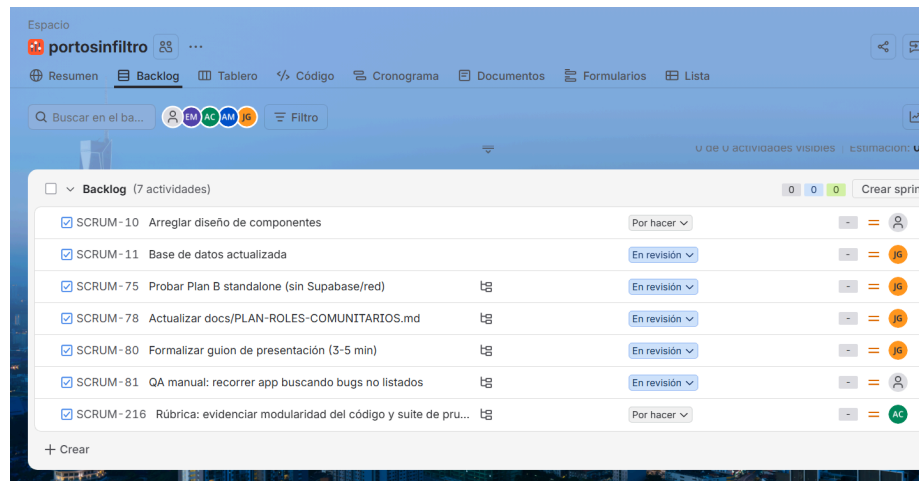


Figura 12. Backlog del proyecto en Jira.

A.6 Trazabilidad de las evidencias

La siguiente tabla relaciona cada evidencia con los requisitos y apartados del documento que respalda.

Figura	Evidencia	Respalda
Figura 7	Aplicación en producción	RF-06, RF-16 y RF-17; criterios de éxito 4 y 8; enlace de producción de la sección 23.
Figura 8	Integración continua en GitHub Actions	RNF-11; entregable «Flujo de integración continua» de la sección 17.
Figura 9	Pruebas automatizadas del backend	RNF-05 (limitación de peticiones) y RNF-10; entregables de pruebas de la sección 17.
Figura 10	Pruebas automatizadas del frontend	RF-07 (Realtime) y RF-10 y RF-11 (estado comunitario); RNF-10.
Figura 11	Lista de actividades en Jira	Sección 6 (metodología) y sección 7 (roles del equipo).
Figura 12	Backlog del proyecto en Jira	Sección 6: organización del trabajo y seguimiento por estados.